Changelog: v20 → v21

Survival search, trap pricing and the long-worker escape slot

Summary

v21 adds a depth-first *survival search* (`survival.hpp`): for each candidate move, it asks whether our dragon can keep moving for its own length plus two more turns, playing out the engine's movement rules exactly. The result feeds the safety filter (`critical_ok`) and the position score. Two supporting changes came out of testing: traps are now priced by whether a split-escape is actually possible, and workers of length ≥ 8 may use the reserved escape slot at the unit cap. No other behaviour changed. `coordination.hpp`, `route.hpp` and `helper.hpp` are byte-identical to v20.

File	Change
`survival.hpp`	**New** (207 lines): the survival DFS
`main.cpp`	Config constants, `Candidate` fields, hook, safety filters, escape slot, include, debug log
`scoring.hpp`	`trap_cost`, new `position_value` branches
`explore.hpp`	Pearl sprints use the survival search
`endgame.hpp`	Comment corrected (death yield)
`coordination.hpp`, `route.hpp`, `helper.hpp`	Unchanged
`sim/probe_aggr.py`	Accidental self-death metrics
`sim/probe_trapped.py`	**New**: diagnoses no-legal-move deaths

1 Movement rules the search is built on

These were confirmed before writing the search and are what `surv_dfs` simulates:

1. **Self-collision includes the tail**, and is checked *before* the tail advances. Within a multi-step move, each step is checked against the body as the previous step left it.
2. **Dragons act one at a time**, and each sees the actual board at its turn.
3. **Sprints** check collisions at every step and remove one extra tail segment per extra step.
4. **Pay before eat**: the validity check for a step happens before that step's pearl is eaten, so a pearl does not save a dragon on its death step.
5. **Death yield**: a dragon of length L always drops $\lceil L/2 \rceil$ pearls.
6. **No simultaneous entry**: two dragons never move into a tile at the same time.

The game update (queens with id 0 and 1, the queen-length win condition after 500 rounds, and $\lceil L/4 \rceil$ free moves before a sprint costs length) is **not** implemented in v21, as requested.

2 New file: `survival.hpp`

Included inside `ReplayDerivedBot`, before `scoring.hpp`.

2.1 What it computes

```
struct SurvivalResult {
    int depth = -1; // most moves survived on any path (-1: not searched)
    int target = 0;
    bool complete = false; // every path searched (budget did not cut it short)
    bool ok() const { return depth >= 0 && depth >= target; }
    bool dead() const { return depth >= 0 && complete && depth * 4 < target * 3; }
};
```

- **Target depth**: $\min(L + \texttt{DFS_EXTRA_DEPTH},\ \texttt{DFS_MAX_DEPTH}) = \min(L + 2, 20)$, where L is our length after the candidate move. Surviving L moves means our whole current body has moved off its tiles; the +2 is margin.
- **ok**: some path reached the target.
- **dead**: the search finished in full and the best path is short of 3/4 of the target. This is a trap.
- Anything else (including a budget cut) is "unknown", and the old v20 heuristics decide.

2.2 State (new members)

- `other_free_at_`: per tile, the first of our moves at which another dragon's segment has gone (`INF`: never).
- `surv_occ_`: our body during the search (occupancy counts).
- `surv_eaten_`: pearls eaten along the current path, so the same pearl is not eaten twice.
- `surv_nodes_`, `surv_turn_nodes_`: per-candidate and per-turn node counters.

2.3 `prepare_survival()`: once per turn

Called from `play_turn` right after `choose_goal()`. It resets the turn budget and fills `other_free_at_`:

1. A tile with a **remembered** (out-of-sight) body blocks for $\texttt{DFS_REMEMBERED_STEPS} = 3$ steps.
2. **Our own unplaceable segments** (`own_idx_ == -2`, never seen since birth) free after $\max(1,\ L - \texttt{known_body_})$ moves, since they are in the unknown tail.
3. **Every visible part of another dragon** is a wall (`INF`) by default.
4. For each **visible head**, the body is traced (`trace_body`). Segment i frees after $\max(\texttt{DFS_OTHER_MIN_WAIT},\ \mathit{est} - i + 1)$ of our moves, where est is that dragon's estimated length. Each dragon moves once between two of ours; the +1 covers it eating (its tail then stays); and nothing frees before our second move. Segments whose head we cannot see stay walls.
5. Each visible head's **non-reverse next tiles** get a value of at least 2. That head may step there before our next move, so they are not counted on for our first step. Where heads go after that is left to the fight search.

2.4 `survival(pm, heading)`: per candidate

- Builds our body after the move, head first: `pm.path` reversed, then `own_trail_`, truncated to `pm.length`. Any remaining length is an `unknown` count of tail segments we cannot place.
- Marks the body in `surv_occ_`, runs `surv_dfs`, unmarks it, and returns the depth and completeness.
- If the per-turn budget is already spent, it returns "not searched" (`depth = -1`).

2.5 `surv_dfs`: the search

- Stops with success at the target. It stops with `cut = true` when either budget runs out: `DFS_NODE_BUDGET` = 4000 per candidate, `DFS_TURN_BUDGET` = 40000 per turn.
- **Children** are the (up to three) non-reverse directions whose destination is passable and known. The tile must also hold none of our body (tail included, per rule 1) and have `other_free_at_` $\leq$ the step number. Kelp blocks, the map wraps, and known portals are followed. Unknown portals are not taken, since we cannot see where they lead.
- **Ordering**: children are sorted by *openness*, the count of free neighbours at the next step, most open first. In open water the first path reaches the target, so the search is cheap; the budget only bites in tight spots. This is a manual insertion sort over at most three elements (see §7).
- **Moving**: the head is pushed. If the tile has a pearl seen this round that is not already eaten on this path, we grow and the tail stays. Otherwise an unknown tail segment is consumed if any remain, else the real tail tile is popped.
- **Backtracking** undoes every change: the head push, the tail pop, the unknown count and the eaten marks.
- Records the deepest depth reached in `best`.

3 `main.cpp`

3.1 Header comment

```
// v20: scoring in segments, one goal per turn, fight lookahead (scoring.hpp).
// v21: survival search over our own next moves (survival.hpp); traps priced by
// whether we could split out; long workers may use a reserved escape slot.
```

3.2 New configuration

```
#ifndef LONG_WORKER_ESCAPE
#define LONG_WORKER_ESCAPE 8
#endif
constexpr int LONG_WORKER_ESCAPE_LENGTH = LONG_WORKER_ESCAPE;

// v21: survival search
constexpr int DFS_EXTRA_DEPTH = 2; // target = length + this ...
constexpr int DFS_MAX_DEPTH = 20; // ... capped here
constexpr int DFS_NODE_BUDGET = 4000; // per candidate
constexpr int DFS_TURN_BUDGET = 40000; // per turn
constexpr int DFS_REMEMBERED_STEPS = 3; // remembered body blocks this long
constexpr int DFS_OTHER_MIN_WAIT = 2; // no other segment frees before this
constexpr double P_DFS_SHORT = 0.9; // trap probability per unit shortfall
```

`LONG_WORKER_ESCAPE` sits after `ESCAPE_UNIT_RESERVE = 2`. Its comment records why it exists: boxed-in workers at the unit cap used to die whole, about 140 deaths of dragons of length 10+ per 200 rounds on a big map. Compiling with `-DLONG_WORKER_ESCAPE=1000` turns the feature off.

3.3 `Candidate` fields

```
int dfs_depth = -1, dfs_target = 0; bool dfs_complete = false;
bool dfs_ok() const { return dfs_depth >= 0 && dfs_depth >= dfs_target; }
bool dfs_dead() const { return dfs_depth >= 0 && dfs_complete
                            && dfs_depth * 4 < dfs_target * 3; }
```

3.4 Turn hook and include

- `play_turn`: `prepare_survival();` is called immediately after `choose_goal();`.
- `##include "survival.hpp"` is added before `scoring.hpp`.

3.5 Escape slot policy (`escape_slot_available`)

Previously a worker always had to leave `ESCAPE_UNIT_RESERVE` = 2 unit slots free before it could split-escape. Now:

```
reserve = king ? 0
       : (valuable_now_ || length >= LONG_WORKER_ESCAPE_LENGTH) ? 1
       : ESCAPE_UNIT_RESERVE;
```

A worker of length ≥ 8 may use the second-to-last slot to split out of a trap, like a valuable dragon. Losing 8+ segments whole costs far more than one extra unit near the cap.

3.6 `evaluate_single`

- After the room/space facts are gathered, `survival(pm, direction)` runs and the result is stored in `dfs_depth`, `dfs_target` and `dfs_complete`.
- **Safety filter**: the room part of `critical_ok` is now

  ```
  room_ok = dfs_ok()
        || (!dfs_dead() && exits > 0
            && space >= pm.length + unknown_body() + 2 && !trap);
  ```

  A move the search proves survivable is safe even when the flood-fill heuristics call it tight. A move the search proves a trap is unsafe even when they look fine. Unknown falls back to v20. The other conditions are unchanged: known destination, no unknown portal, no head trade, no occupied exit, and for valuable dragons no hard threats or blind portal.

3.7 Escape sprints (`add_escape_sprints`)

Each escape sprint runs `survival(pm, steps.back())` from its final position, and

```
critical_ok = hard_threats == 0
          && (dfs_ok() || (!dfs_dead() && exits > 0
                           && space >= need + 2 && !trap));
```

3.8 Debug log

The `CAND` debug line now ends with `dfs <depth> <target> C|-`, where `C` means the search was complete.

4 `scoring.hpp`

4.1 `trap_cost`

In v20 a trap was priced as if a split-escape were always available (2 segments plus a unit). At the unit cap it is not, and the dragon dies whole, so traps were underpriced by about 3×. Now:

```
int const L = controller_.get_length();
bool const can_escape = L >= 4 && escape_slot_available()
                     && controller_.can_split(L - 2);
return can_escape ? seg(2.0 + Config::UNIT_VALUE_OURS) : death_cost();
```

4.2 `position_value`

New branches after the pocket-plan case:

- `dfs_ok()`: the room is proven. Only the `P_ONE_EXIT` risk is charged when `exits == 1` (a single exit can still be taken by someone else). The space and dead-end risks are skipped.
- **Searched in full but short**: with $shortfall = (target - depth)/target$,

  ```
  v -= risk(P_DFS_SHORT * shortfall, trap_cost());
  ```

  The penalty grows smoothly with how far short the best path falls, priced at the new `trap_cost`.
- **Not searched, or cut short by the budget**: the v20 exits/space/dead-end risks, unchanged.

5 `explore.hpp`

`add_pearl_sprints` runs `survival(pm, steps.back())` from each sprint's end, and

```
critical_ok = (dfs_ok() || (!dfs_dead() && exits > 0
                            && space >= need + 2 && !trap))
          && (!valuable || hard == 0);
```

6 `endgame.hpp`

Comment only. "Full yield" was misleading, because every death yields $\lceil L/2 \rceil$:

```
// Every death leaves ceil(L/2) pearls on the body's tiles; dying on our own
// neck here (rather than in a trade) puts them where the king will eat them.
```

7 How the search got here

1. **First version (optimistic).** Other dragons' tails freed at step 1 and other heads were ignored. Since `dfs_ok()` drops the `exits == 0` risk, dragons stepped onto tiles with no free neighbour. It went 5–3 against v20 and 2/2 against the baseline, but the failure mode was real.
2. **Fix: cautious timing.** Added `DFS_OTHER_MIN_WAIT = 2` and the $est - i + 1$ eating margin, and blocked other heads' next tiles for our first step.
3. **Fix: trap pricing.** Diagnosis showed nearly all self-deaths were `TRAPPED_SELF` (no legal move) at ≥ 62 units, where workers cannot split-escape. `trap_cost` was changed to check `escape_slot_available() && can_split`.
4. **Escape slot.** Even correctly priced traps still happen at the cap, so long workers were allowed the reserve slot.
5. **Compiler warning.** `std::sort` on `std::array<Child,3>` triggered a GCC `-Warray-bounds` false positive. It was replaced with a manual insertion sort.

8 Tooling (`sim/`)

- `probe_aggr.py`: new *accidental self-death* metric. It counts `hit_self` deaths of length ≥ 3 that are not deliberate (`FEED`, `REMNANT_CLEAR`), broken down by phase (early < 380 / late), cause (`TRAPPED_SELF` vs moved into self) and length bucket. It also counts `GOAL_*` log lines.
- `probe_trapped.py` (**new**): for each `TRAPPED_SELF` death, it reports the previous turn's chosen candidate (dfs depth/target, exits, space), the team's unit count, and what walls surround the head (ally body, ally head, own body, kelp, enemy).

9 Results

9.1 No-legal-move deaths (200 rounds, seed 20, vs baseline)

Version	Trapped deaths	of which length 10+
v20	318	157
v21, cautious DFS	297	–
v21, + trap pricing	270	–
v21, + escape slot (final)	238	92

Walls around trapped heads were mostly allies (ally body 463–499, ally head 86–111), then own body (∼450), kelp (∼120) and enemies (∼25). Crowding at the unit cap, not enemy play, is the main cause.

9.2 Games

- v21 without the escape slot vs v20: 2–2.
- v21 final vs v20: **4–2**, with accidental self-deaths about 27% lower on big maps.

9.3 Other checks

- Scenario tests: unchanged, all pass.
- Coil stress test (`stress_box.py`): same failures as v20. These are blind splits caused by the unknown body, which the search cannot see either.
- Maximum turn time: 5–13 ms.

10 Known limitations and not yet done

- **Sprint rule update** ($\lceil L/4 \rceil$ free moves): not implemented. Sprint costs still follow the old rule.
- **Queens / win condition**: not implemented.
- The search covers *our* moves only. Other dragons are modelled as bodies that drain away, plus a one-step head reach.
- Sprints are not explored inside the search (single moves only), so it can under-count escapes.
- Pearls are only counted if seen this round. Remembered pearls are ignored, which is conservative.
- Crowding at the unit cap remains the largest source of trapped deaths.
- Planned but not started: panic fallbacks, contest ETAs.